

Subiecte Acadnet Interoperabilitate Software

Etapă Interjudețeană 2018

1. CLI Calculator

Mariei îi face plăcere să demonstreze teoreme la matematică și să rezolve integrale, însă are totuși o mică problemă cu operațiile de bază... mai exact, se încurcă la cele mai simple calcule. Totuși, ea a descoperit o cale de a utiliza cunoștințele de programare nou dobândite în folosul său și a scris un program care să o ajute.

Acesta încă nu este complet - momentan suportă doar operațiile unare "inc", "dec", "abs", "shift", "pow2" și operațiile binare "+", "-", ".*" și "/". Programul începe prin a cere o valoare inițială pe care o va modifica pe măsură ce se introduce câte o operație (dacă aceasta este binară, pe următoarea linie trebuie introdus și al doilea operand). Operațiile invalide (cu nume greșite) vor fi omise. Programul se termină la comanda quit sau la încercarea de a împărți cu zero.

Maria, încântată de programul ei, i l-a arătat unui coleg, încă acesta i-a atras atenția că există o eroare și că rezultatele nu sunt întotdeauna cele așteptate. Ea a încercat să descopere cauza problemei, însă, obosită și cam dezorientată de numărul mare de linii de cod, nu a reușit să o depisteze.

Tu poți să o ajuți ?

Exemplu de input și rezultat așteptat:

```
IN
3
*
3
*
0
dec
inc
-
-1
quit
OUT
```

```
-----Welcome to the COMMAND-LINE CALCULATOR v1.0!-----
-----
```

```
quit          exit program
menu          display this menu
abs           absolute value
inc           increment value
dec           decrement value
pow2          raise to the power of 2
```

```

shift          shifts the bits to the left by one
+              +
-              -
*              *
/              integer division
-----

```

```

Enter the initial value for your variable: Enter command: Enter the second
operand: Result => 9
Enter command: Enter the second operand: Result => 0
Enter command: Result => -1
Enter command: Result => 0
Enter command: Enter the second operand: Result => 1
Enter command:

```

```

#include <iostream>
#include <cmath>
#include <string>
#include <sstream>

```

```

#define NONE 0
#define WELCOME_BANNER "-----Welcome to the COMMAND-LINE
CALCULATOR v1.0!-----"
#define INVALID_OPERATION_ERROR "ERROR: Operation not defined."
#define DIV_BY_ZERO_ERROR "ERROR: Divide by zero."
#define FIRST_VAR_PROMPT "Enter the initial value for your variable:
"
#define SECOND_VAR_PROMPT "Enter the second operand: "
#define COMMAND_PROMPT "Enter command: "
#define RESULT_STR "Result => "

#define QUIT "quit"
#define MENU "menu"
#define ABS "abs"
#define INC "inc"
#define DEC "dec"
#define POW2 "pow2"
#define SHIFT "shift"
#define ADD "+"
#define SUB "-"
#define MULT "*"
#define DIV "/"

#define DQUIT "exit program"
#define DMENU "display this menu"
#define DABS "absolute value"

```

```

#define DINC          "increment value"
#define DDEC          "decrement value"
#define DPOW2         "raise to the power of 2"
#define DSHIFT        "shifts the bits to the left by one"
#define DADD          "+"
#define DSUB          "-"
#define DMULT          "*"
#define DDIV          "integer division"

#define abs(a)         abs(a)
#define inc(a)         ++(a)
#define dec(a)         --(a)
#define pow2(a)        (a) * (a)
#define shift(a)       (a) << 2
#define add(a, b)      (a) + (b)
#define sub(a, b)      (a) - (b)
#define mult(a, b)     (a) * (b)
#define div(a, b)      (a) / (b)

#define IS_UNARY_OP(op)    (op) < 100 && (op) >= 0
#define IS_BINARY_OP(op)  (op) >= 100 && (op) >= 0

enum OpCode {
    OP_ABS = 0,
    OP_INC,
    OP_DEC,
    OP_POW2,
    OP_SHIFT,

    OP_ADD = 100,
    OP_SUB,
    OP_MULT,
    OP_DIV,

    OP_QUIT = -10,
    OP_MENU = -11,

    OP_INVALID = -1,
};

int set_initial_value() {
    int var;
    std::cout << FIRST_VAR_PROMPT;
    std::cin >> var;
    return var;
}

```

```

#define PRINT_MENU_ENTRY(cmd) std::cout << cmd << "\t\t" << D##cmd << std::endl;
void show_menu() {
    std::cout << "-----\n";
    PRINT_MENU_ENTRY(QUIT);
    PRINT_MENU_ENTRY(MENU);
    PRINT_MENU_ENTRY(ABS);
    PRINT_MENU_ENTRY(INC);
    PRINT_MENU_ENTRY(DEC);
    PRINT_MENU_ENTRY(POW2);
    PRINT_MENU_ENTRY(SHIFT);
    PRINT_MENU_ENTRY(ADD);
    PRINT_MENU_ENTRY(SUB);
    PRINT_MENU_ENTRY(MULT);
    PRINT_MENU_ENTRY(DIV);
    std::cout << "-----\n";
}

int execute_operation(OpCode op, int arg1, int arg2) {
    //std::cout << op << std::endl;
    std::stringstream ss;
    switch (op) {
        case OP_ABS:
            ss << abs(arg1);
            break;
        case OP_INC:
            ss << inc(arg1);
            break;
        case OP_DEC:
            ss << dec(arg1);
            break;
        case OP_POW2:
            ss << pow2(arg1);
            break;
        case OP_SHIFT:
            ss << shift(arg1);
            break;
        case OP_ADD:
            ss << add(arg1, arg2);
            break;
        case OP_SUB:
            ss << sub(arg1, arg2);
            break;
        case OP_MULT:
            ss << mult(arg1, arg2);
            break;
    }
}

```

```

        case OP_DIV:
            if (arg2 == 0) {
                std::cout << DIV_BY_ZERO_ERROR;
                exit(-1);
            }
            ss << div(arg1, arg2);
            break;
        default:
            break;
    }

    ss >> arg1;
    return arg1;
}

```

```

OpCode get_opcode(std::string op) {

```

```

    if (op == POW2) {
        return OP_POW2;
    } else if (op == SHIFT) {
        return OP_SHIFT;
    } else if (op == ABS) {
        return OP_ABS;
    } else if (op == INC) {
        return OP_INC;
    } else if (op == DEC) {
        return OP_DEC;
    } else if (op == ADD) {
        return OP_ADD;
    } else if (op == SUB) {
        return OP_SUB;
    } else if (op == MULT) {
        return OP_MULT;
    } else if (op == DIV) {
        return OP_DIV;
    } else if (op == QUIT) {
        return OP_QUIT;
    } else if (op == MENU) {
        return OP_MENU;
    } else {
        return OP_INVALID;
    }
}

```

```

OpCode get_command() {

```

```

    OpCode op;
    std::string command;

    do {
        std::cout << COMMAND_PROMPT;
        std::cin >> command;
        op = get_opcode(command);
        if (op == OP_INVALID) {
            std::cout << INVALID_OPERATION_ERROR << std::endl;
        }
    } while (op == OP_INVALID);

    return op;
}

int main() {
    int var;
    int var2;
    OpCode op;

    std::cout << WELCOME_BANNER << std::endl;
    show_menu();
    var = set_initial_value();

    while(true) {
        op = get_command();

        if (op == OP_QUIT) {
            break;
        }

        if (op == OP_MENU) {
            show_menu();
            continue;
        }

        if (IS_UNARY_OP(op)) {
            var = execute_operation(op, var, NONE);
        } else if (IS_BINARY_OP(op)) {
            std::cout << SECOND_VAR_PROMPT;
            std::cin >> var2;
            var = execute_operation(op, var, var2);
        }

        std::cout << RESULT_STR << var << std::endl;
    }
}

```

```
    return 0;
}
```

2. Statistica mediilor

Răzvan este elev la liceul din Nicăieri. El dorește să facă o statistică despre mediile celor mai buni dintre colegii lui. Pentru asta, el are la dispoziție mediile tuturor colegiilor și își dorește să calculeze media celor mai mari dintre ele. Pentru calculul mediei, Răzvan va considera doar jumătate din colegi, anume cei cu mediile cele mai mari.

El știe cu siguranță că în clasa lui se afla un număr par de elevi, deci sigur nu-ți va fi greu să descoperi ce note trebuiesc luate în considerare. Pentru asta el a scris un program care citește de la tastatură numărul de studenți din clasă urmat de o lista cu mediile lor.

La ieșirea standard se va afișa media calculată, rotunjită în jos la cel mai apropiat întreg.

Input:

numărul de studenți - o lista cu mediile colegiilor (neordonata)

Output:

statistica cerută (rotunjită în jos la cel mai apropiat întreg)

```
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

template <typename Iterator>
void bubble_sort(Iterator begin, Iterator end) {
    bool swapped;
    do {
        swapped = false;
        for (Iterator i = begin + 1; i != end; ++i) {
            typename std::iterator_traits<Iterator>::value_type& val0 = *i;
            typename std::iterator_traits<Iterator>::value_type& val1 =
                *(i - 1);
            if (val0 < val1) {
                std::swap(val1, val0);
                swapped = true;
            }
        }
    } while (swapped);
}
```

```

        }
    } while (swapped);
}

int main() {
    size_t n, i;
    int temp;

    cin >> n;
    vector<int> numbers(n);
    for (i = 0; i < n; i++) {
        cin >> temp;
        numbers[i] = temp;
    }

    bubble_sort(numbers.begin(), numbers.end());

    int sum = 0;
    int num = 0;
    for (i = n - 1; i >= n / 2; i--) {
        num++;
        sum += numbers[i];
    }

    cout << sum / temp << endl;
}

```

3. Fabrica de jucării

La o fabrică de jucării, se produc cutii cu jucării. Fiecare cutie, în mod normal, are un cod de producție (număr de ordine). În fiecare zi, în toate cutiile, se pun un număr egal de jucării.

Din păcate, astăzi banda de producție s-a dereglat, iar în cutii sunt numere aleatoare de jucării. Muncitorii de la fabrică trebuie să repare problema așa că s-au decis să o repare manual.

Pentru toate cutiile de jucării produse azi, muncitorii fac următoarea operație:

Mută din cutia cu cele mai multe jucării, în cutia cu cele mai puține jucării, un număr de jucării astfel încât în cutia cu cele mai puține să fie după această operație atâtea jucării câte trebuiau puse în acea zi.

Rezolvați problema muncitorilor scriind în **fișierul de ieșire** (jucarii.out) următoarele date:

- Pe prima linie din fișier: Numărul de mutări ce trebuie efectuate

- Pe următoarele linii, pentru fiecare operație: (Codul cutiei din care se mută) (Numărul de jucării mutate) (Codul cutiei în care se mută)

Date de intrare:

- Pe prima linie: Numărul de cutii produse de fabrică astăzi
- Pe următoarea linie: Pentru fiecare cutie, în ordinea codurilor cutiilor, numărul de jucării din fiecare cutie

Precizări:

- În fiecare zi, fabrica produce între 0 și 1000 de cutii de jucării
- Este posibil ca jucăriile din cutii să nu poată fi repartizate în mod egal în toate cutiile. În acest caz în fișierul de ieșire se va tipări doar “-1”.
- În fiecare zi, cutiile se numerează de la 1 la n, unde n este numărul de cutii produse în acea zi.
- În fiecare cutie, indiferent de zi (adică inclusiv astăzi), sunt maxim 1000 de jucării

Exemplu:

jucarii.in

5

4 7 1 2 6

jucarii.out

2

2 3 3

5 2 4

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```
struct Cutie {
    int index;
    int nr_jucarii;
};
```

```
struct Mutare {
    int sursa;
    int destinatie;
    int jucarii;
};
```

```
bool rule(Cutie a, Cutie b) {
    return a.nr_jucarii < b.nr_jucarii;
```

```

}

int main() {
    int nr_cutii;
    cin >> nr_cutii;

    if (nr_cutii == 0 || nr_cutii == 1) {
        cout << "0" << endl;
    } else {
        struct Cutie cutii[1005];
        int medie_jucarii = 0;

        for (int i = 0; i < nr_cutii; i++) {
            cin >> cutii[i].nr_jucarii;
            medie_jucarii += cutii[i].nr_jucarii;
            cutii[i].index = i;
        }
        medie_jucarii /= nr_cutii;

        struct Mutare mutari[1005];

        int count = 0;
        sort(cutii, cutii + nr_cutii, rule);

        while (medie_jucarii - cutii[0].nr_jucarii > 0) {

            mutari[count].jucarii = cutii[nr_cutii - 1].nr_jucarii - medie_jucarii;
            cutii[0].nr_jucarii += mutari[count].jucarii;
            cutii[nr_cutii - 1].nr_jucarii -= mutari[count].jucarii;

            mutari[count].sursa = cutii[nr_cutii - 1].index;
            mutari[count].destinatie = cutii[0].index;

            count++;
            sort(cutii, cutii + nr_cutii, rule);
        }

        cout << count << endl;

        for (int i = 0; i < count; i++) {
            cout << mutari[i].sursa << " " << mutari[i].jucarii << " ";
            cout << mutari[i].destinatie << endl;
        }
    }

    return 0;
}

```

```
}
```

4. Weird Sort

Daniela s-a gandit cum ar putea sorta cat mai ciudat elementele unei matrici, in functie de paritatea liniei, astfel ca ea doreste sa sorteze elementele de pe liniile pare in ordine descrescatoare, iar elementele de pe liniile impare, in ordine crescatoare. Din păcate, programul nu funcționează corect și este rolul vostru să-l reparați.

Precizări:

- Liniile si coloanele sunt indexate de la 0, nu de la 1!
- Porniți rezolvarea folosindu-vă de codul sursă oferit, ce conține funcțiile main si weirdSort. Nu este nevoie sa modificați scheletul de parsare (funcția main).
- Rezolvarea trebuie făcută folosindu-se maxim 5 modificări. Se consideră a fi o modificare orice linie ce difera prin cel puțin un caracter de linia din codul sursă original.

Input file name: input.txt Time limit: 1 s Real time limite: 5 s Memory limit: 64 M

Exemplu:

Input	Output
3	3 2 1
1 3 2	1 3 5
5 1 3	6 3 1
6 3 1	

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <vector>

#define INPUT_FILENAME "input.txt"
#define OUTPUT_FILENAME "output.txt"

using namespace std;

void weirdSort(vector<vector<int>> &matrix) {
    int size = matrix[0].size();
    int i, j;
    bool sw = true;
```

```

    for (i = 0; i < size; i++) {
        sw = true;
        while (sw) {
            sw = false;
            for (j = 0; j < size - 1; j++) {
                if (i % 2) {
                    if (matrix[i][j] > matrix[i][j + 1]) {
                        sw = true;
                        swap(matrix[i][j], matrix[i][j + 1]);
                    }
                } else {
                    if (matrix[i][j] < matrix[i][j + 1]) {
                        sw = true;
                        swap(matrix[i][j], matrix[i][j + 1]);
                    }
                }
            }
        }
    }

    for (i = 0; i < size; i++) {
        for (j = 0; j < size; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
}

int main() {
    vector<vector<int>> matrix;
    ifstream inputFile;
    ofstream outputFile;

    inputFile.open(INPUT_FILENAME);

    if (!inputFile.good()) {
        cout << "Failed to open" << INPUT_FILENAME << endl;
        exit(-1);
    }

    int size;
    inputFile >> size;

    matrix.reserve(size);

    for (int i = 0; i < size; i++) {

```

```

        matrix[i].reserve(size);
    }

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int x;
            inputFile >> x;
            matrix[i].push_back(x);
        }
    }

    inputFile.close();

    weirdSort(matrix);

    outputFile.open(OUTPUT_FILENAME);
    if (!outputFile.good()) {
        cout << "Failed to open" << OUTPUT_FILENAME << endl;
        exit(-1);
    }

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            outputFile << matrix[i][j] << " ";
        }
        outputFile << endl;
    }

    outputFile.close();
    return 0;
}

```